ISLAMIC UNIVERSITY OF TECHNOLOGY
A Subsidiary Organ of The Organisation of Islamic Cooperation (OIC)

## Proposal Report

# Assignment Name

## Software Project Lab-1

SWE 4304

Third Semester, Session 2023-2024

## Submitted to

Samia Nawsheen, Junior Lecturer, CSE

## Submitted by

Sheikh Alimun Alahi
230042115
Rakibur Rahman Ananta
230042128
Irfan Sadik Chowdhury
230042147

## Date of Submission

9 March, 2024

## Project Overview :

The **Airport Management System** is a terminal-based application that simulates the essential operations of an airport. It is built with a modular OOP structure and relies entirely on file-based storage instead of internet APIs or databases. To manage passengers without airline systems, the airport follows a different ticketing model, where it directly sells tickets, stores passenger data, and prepares flights for transportation.

The system is organized into clear functional modules. **Flight Management** maintains all flight details, tracks flight status, checks operational conditions, and determines when a flight can begin boarding based on weather, gate availability, and runway readiness. **Gate Allocation** and **Runway Allocation** systems manage the assignment of these resources to flights, keeping track of their status and resolving conflicts.

Passenger handling is separated into **Passenger Management**, which stores all passenger records in a static list, supports check-ins, and issues boarding passes, and a **Booking** module that allows customers to select flights, confirm availability, and receive an ASCII-format ticket upon confirmation.

To add dynamic behavior, the **Weather** module generates random conditions, and the upcoming Simulation Engine and Chaos Mode introduce time-based updates and unpredictable events that impact scheduling and clearance.

The system interface is organized through a **View Menu**, while **Login Management** controls access to different modules using a credentials file. Altogether, the project presents a clean, CLI-based simulation that demonstrates OOP concepts, resource allocation, event handling, and airport workflow in a simplified but functional manner.

## Motivation behind the Project :

Airports are one of the most complex systems we interact within everyday life. Every minute, flights are taking off, landing, being delayed, boarding passengers, releasing gates, requesting runways and handling unexpected issues. Behind that smooth flow lies a constant battle against limited resources, strict timings, and unpredictable events.

This level of complexity makes airports a perfect environment to test our engineering skills. Managing a flight is not just data entry—each decision affects every other part of the system. A gate assigned at the wrong time can delay multiple flights. A weather issue can cause a chain reaction. A runway conflict can halt the entire operation. These real-world constraints motivated us to choose the airport as our project theme.

At the same time, we also faced our own challenge:
No database, no internet, and only standard libraries**.**
This limitation pushed us toward a completely self-contained design. We had to rethink how to store passengers, flights, and bookings without relying on external airline systems. That is why we adopted a train-style ticketing model, where the airport itself sells tickets, keeps passenger lists, verifies check-ins, and manages flights end-to-end.

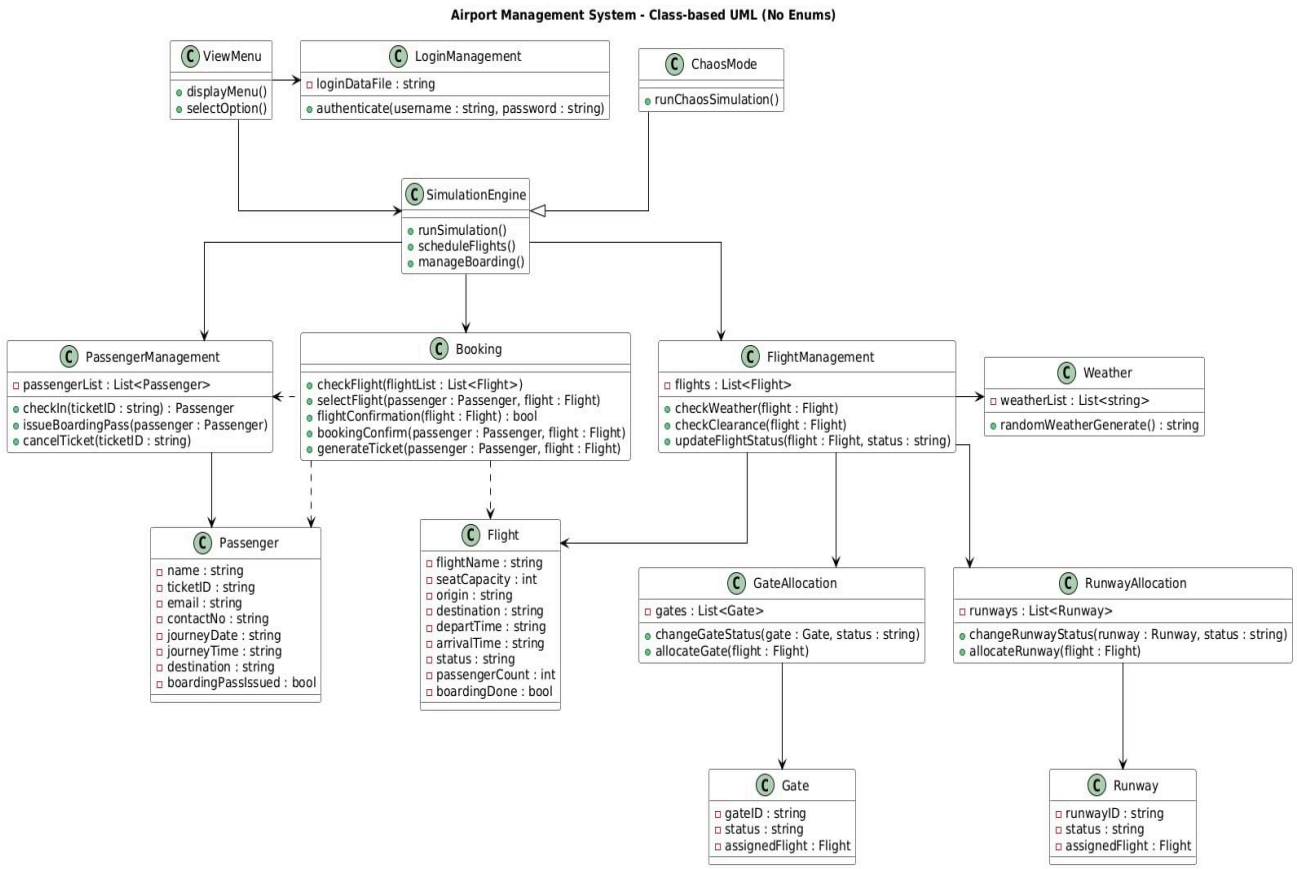
Instead of building a simple CRUD application, we wanted to create something alive—something that moves with time, reacts to events, and gives the user real responsibility. Our simulation engine, our resource managers, and even our Chaos Mode are all designed to bring the airport to life inside the terminal.

In short, our motivation comes from two places:

- The real-world challenge of scheduling, resource allocation, and event handling.
- The engineering challenge of building a complete system from scratch, using only our own logic, data structures, and object-oriented design.

This project is not just about managing flights—it's about understanding how complex systems operate under constraints, and proving that with careful planning and solid programming fundamentals, we can build an entire airport system inside a command line.

# Class Diagram




This UML shows an Airport Management System where a central SimulationEngine controls flight operations.

- LoginManagement and ViewMenu handle user access and interaction.
- PassengerManagement and Booking manage passengers, tickets, and flight bookings.
- FlightManagement oversees flights and reacts to Weather conditions.
- GateAllocation and RunwayAllocation assign gates and runways to flights.
- ChaosMode simulates emergency or abnormal situations.

Overall, it models how passengers, flights, gates, runways, and system controls interact in an airport.

## Key Features :

### 1. Secure & Structured Access Control

- Headless CLI Interface: A fully text-based command-line interface that serves as the user's main dashboard.
- File-Based Login Management: Each management role (Flight, Gate, Booking) requires authentication. Usernames and passwords are validated through .txt files, ensuring secure access without relying on external databases.
- Centralized View Menu: A main navigation hub that directs users to specific subsystems—Flight, Gate, Runway, Passenger and Booking—based on the logged-in user's permissions.

### 2. Comprehensive Flight Management

- Flight Lifecycle Tracking: Flights progress through multiple stages: Scheduled → Allocated → Boarding → Takeoff/Landed.
- Automated Pre-Flight Clearance: A "Safety First" logic engine authorizes boarding or takeoff only when all conditions are satisfied :
    - Gate Check: A gate must be allocated.
    - Runway Check: The runway must be free.
    - Weather Check: The weather module must return to a safe condition.
- Detailed Flight Attributes: Tracks seat capacity, origin & destination, date & time, flight status and passenger count.

### 3. Resource & Infrastructure Allocation

- Smart Gate Allocation: Manages a List<Gate> that allows assigning flights to specific gates while tracking gate IDs and occupancy status (Free/Occupied), preventing allocation conflicts.
- Runway Management: A similar List<Runway> structure ensures efficient allocation and release of runways to maintain smooth traffic flow.

## 4. Booking & Passenger Services

- Capacity-Aware Booking: The booking engine lets users select flights based on destination and date and automatically checks availability by comparing passenger count with chosen flight's seat capacity.
- ASCII Ticket Generation: After booking, a neatly formatted ASCII ticket is generated containing all passenger details.
- Passenger Check-In System: Supports fast lookup by name or unique ticket ID using the static passenger list.
- Boarding Pass Issuance: A boarding pass is issued only after a valid check-in, updating the passenger's flight status accordingly.

## 5. Dynamic Simulation Environment

- Dynamic Simulation Engine: A time-driven engine updates flight statuses and schedules dynamically as the simulation progresses.
- Weather Simulation: Random weather conditions generated through Random Weather Generator() directly influence flight clearance decisions.
- Chaos Mode: Introduces unpredictable disruptions—such as delays or emergencies—to test overall management responsiveness and stress handling.

## 6. Reporting & Persistence

- File-Based Storage: All data (flights, passenger lists, login credentials) is stored in text files, providing persistent state management across sessions.
- Performance Reporting: Generates ASCII-formatted reports that display flight schedules, passenger manifests, and daily operational summaries.

## Tools and Technologies

### 1. C++

The primary programming language used to develop the headless CLI application. C++ implements the system's full Object-Oriented architecture, including encapsulation and modular designs for the Flight, Gate, Runway, Passenger and Booking subsystems.

### 2.Visual Studio Code

The Integrated Development Environment (IDE) used for writing, editing, and debugging the project. VS Code helps maintain a clean codebase, organized file structure, and efficient development workflow.

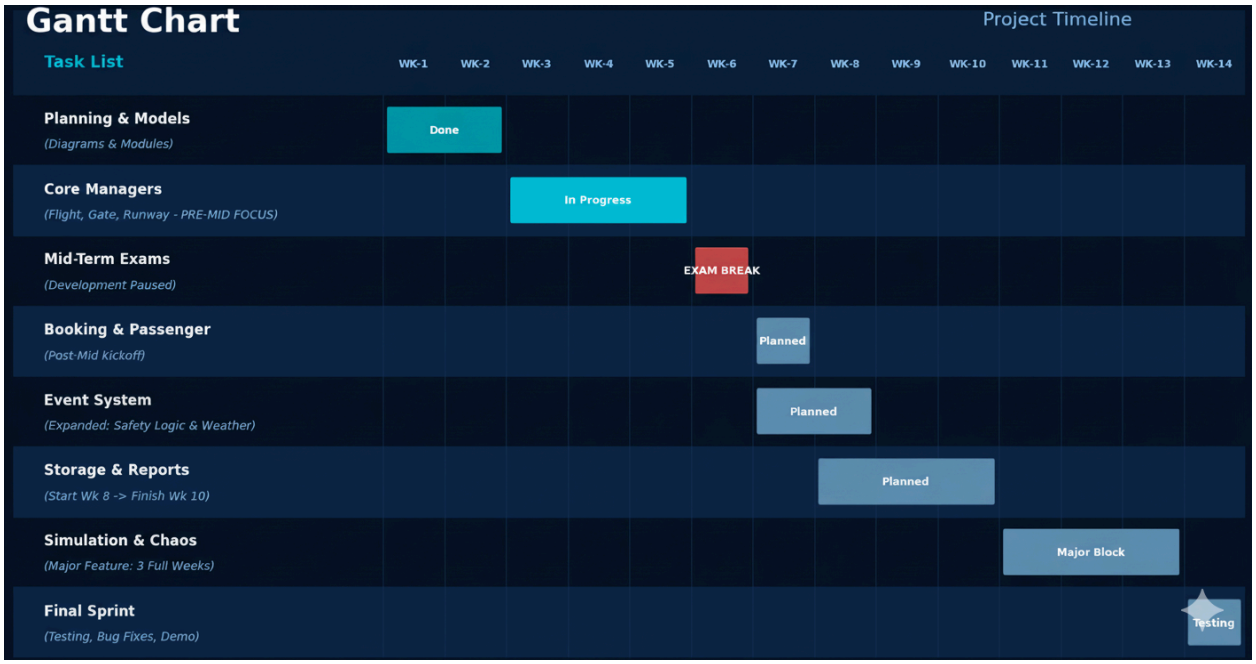
### 3. Standard Library Only

The entire project is built without external libraries or databases. It relies solely on the C++ Standard Library for:

- **Data Structures:** Using queues, lists, and vectors to manage flight scheduling, passenger information, and resource allocation.

- **File I/O:** Handling .txt file reading and writing for login verification and persistent data storage.

# Proposed Timeline




## Action taken on the feedback:

As we did not get any feedback, we are sticking to our initial plan and our supervisor's suggestions.